



app.py

The app.py is the **presentation layer (UI layer)** of the system. It connects users to the database and business logic functions stored in database.py. Below are a comprehensive documentation of every major section and its purpose.

1. Import Statements

```
import streamlit as st
from datetime import datetime, timedelta
import pandas as pd
from database import get_today_summary, get_today_sales,
get_all_products
```

Purpose

These libraries provide the core functionality of the application.

Import	Function
streamlit	Creates the web application interface
datetime	Handles dates and time calculations
timedelta	Calculates date ranges (weekly reports)
pandas	Displays and manipulates tabular data
database functions	Retrieves data from SQLite database



2. Streamlit Page Configuration

```
st.set_page_config(  
    page_title="Omoga Shop Sales System",  
    page_icon="📊",  
    layout="wide",  
    initial_sidebar_state="expanded"  
)
```

Purpose

Configures the application window.

Parameters

Parameter	Function
page_title	Browser tab title
page_icon	Browser tab icon
layout="wide"	Uses full screen width
initial_sidebar_state	Opens sidebar automatically



3. Custom CSS Styling

```
st.markdown("""  
    <style>  
        .metric-card {  
            background-color: #f0f2f6;  
            padding: 20px;  
            border-radius: 10px;  
            margin: 10px 0;  
        }  
        .profit-card {  
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
            color: white;  
            padding: 20px;  
            border-radius: 10px;  
            margin: 10px 0;  
        }  
        .sales-card {  
            background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);  
            color: white;  
            padding: 20px;  
            border-radius: 10px;  
            margin: 10px 0;  
        }  
    </style>  
    """, unsafe_allow_html=True)  
    )
```



Purpose

Adds custom styling beyond Streamlit defaults.

Defined Classes

metric-card

Used for KPI display cards.

```
.metric-card
```

Features:

- Light background
- Rounded corners
- Padding

profit-card

Used for profit-related visual elements.

```
.profit-card
```

Features:

- Purple gradient
- White text
- Professional dashboard appearance

sales-card

Used for sales-related visual elements.

```
.sales-card
```

Features:

- Pink/red gradient
- Enhanced visual emphasis



4. Sidebar Navigation

```
st.sidebar.title("🏠 Shop System")
```

Creates the application's navigation panel.

Radio Menu

```
page = st.sidebar.radio(  
    "Navigate",  
    ["📊 Dashboard", "💵 Add Sale", "📦 Manage Stock", "📄 Reports"]  
)
```

Page	Purpose
Dashboard	View business KPIs
Add Sale	Record customer purchases
Manage Stock	Inventory operations
Reports	Analytics and maintenance

5. Dashboard Module

```
if page == "📊 Dashboard":
```

Main monitoring section.



Dashboard Title

```
st.title("📊 Daily Sales Dashboard")
```

Displays page heading.

Retrieve Summary Data

```
summary = get_today_summary()
```

Obtains:

```
{
    total_sales,
    total_profit,
    total_items_sold,
    transaction_count
}
```

from SQLite.

KPI Cards

```
st.metric()
```

Displays key indicators.

Total Sales

```
KES 45,000
```

Revenue generated today.



Total Profit

KES 12,000

Actual earnings after costs.

Items Sold

320

Total units sold.

Transactions

78

Number of customer purchases.

6. Recent Sales Section

```
sales_data = get_today_sales()
```

Retrieves today's sales.



Convert to DataFrame

```
sales_df = pd.DataFrame(...)
```

Creates a structured table.

Column	Description
ID	Transaction ID
Product	Product sold
Qty	Quantity sold
Price	Unit selling price
Total	Total amount
Profit	Profit earned
Payment	Payment method
Time	Sale timestamp

Format Time

```
pd.to_datetime()
```

Converts timestamps into readable format.

Example:

```
14:30:25
```

Data Table Display

```
st.dataframe()
```

Provides:

- Sorting
- Scrolling
- Responsive layout



7. Current Stock Status

```
products = get_all_products()
```

Loads inventory.

Displayed Fields

Field	Description
Product	Product name
Category	Product category
Buy Price	Supplier cost
Sell Price	Retail price
Quantity	Stock available

Purpose:

Allows manager to monitor inventory levels.

8. Sales Module

```
elif page == "🛒 Add Sale":
```

This is the POS (Point of Sale) system.

9. Session State Initialization

```
st.session_state.shopping_cart
```

Purpose

Maintains cart contents during user interactions.

Without this, cart items would disappear on page refresh.



Stored Information

```
[  
  {  
    product_id,  
    product_name,  
    quantity,  
    total  
  }  
]
```

10. Product Selection

```
st.selectbox()
```

Lists products with stock.

Example:

```
Sugar 2kg (15 in stock)
```

Prevent selling unavailable stock.

11. Quantity Input

```
st.number_input()
```

Restricts:

```
max_value=available_qty
```



12. Price Override

```
price_override
```

Allows special pricing.

Examples:

- Discounts
- Promotions
- Negotiated sales

13. Profit Calculation

```
profit_per_unit = unit_price - buying_price
```

Determines profit per item sold.

14. Add To Cart

```
st.button("🛒 Add to Cart")
```

Function:

Adds selected item into cart.

Logic:

Existing Item

Updates quantity.

New Item

Creates new cart record.



15. Cart Summary

Displays:

Metric	Description
Total Items	Unique products
Total Sale	Revenue
Total Profit	Earnings

16. Payment Method

Cash
M-Pesa
Card
Credit

Used for transaction recording.

Useful for later reporting.

17. Finalize Transaction

`add_sale()`
`reduce_stock()`

Core transaction processing.

add_sale()

Creates transaction record.

reduce_stock()

Updates inventory.



18. Stock Management Module

```
elif page == "📦 Manage Stock":
```

Inventory administration area.

Tab 1: Add Product

Creates new inventory item.

Inputs:

Field	Description
Product Name	Item name
Category	Product group
Buying Price	Supplier cost
Selling Price	Retail price
Quantity	Initial stock

Profit Margin Preview

```
margin = ((selling_price-buying_price)/buying_price)*100
```

Shows expected profitability.

Add Product

```
add_product()
```

Inserts product into database.

Validation:

- Name required
- Buying price > 0
- Selling > Buying



19. Update Stock

Allows modification of:

- Prices
- Categories
- Stock quantity

Uses:

```
update_product()
```

20. View Products

Displays complete inventory.

Delete Product

Uses:

```
delete_product()
```

Removes item permanently.

21. Reports Module

```
elif page == "📄 Reports":
```

Business intelligence section.



22. Daily Report

Uses:

```
get_today_summary()
```

Displays:

- Sales
- Profit
- Items sold
- Transactions

Profit Margin

```
profit_margin =  
(total_profit / total_sales) * 100
```

Measures profitability.

23. Weekly Report

Calculates:

Measures profitability.

```
week_start  
week_end
```

Then retrieves:

```
get_sales_by_date_range()
```

Purpose:

Analyze weekly business performance.



Aggregation

```
groupby('Date')
```

Summarizes:

- Daily sales
- Daily profit
- Daily quantities sold

24. Custom Date Range Report

Allows manager to analyze:

```
1 Jan → 31 Jan
```

or any period.

Uses:

```
get_sales_by_date_range()
```

25. System Maintenance

```
tab4
```

Administrative controls.

Clear Sales

```
clear_all_sales()
```

Deletes:

- Transactions
- Sales history

Keeps:

- Products



Clear Products

```
clear_all_products()
```

Deletes:

- Inventory

Keeps:

- Sales records

Complete Reset

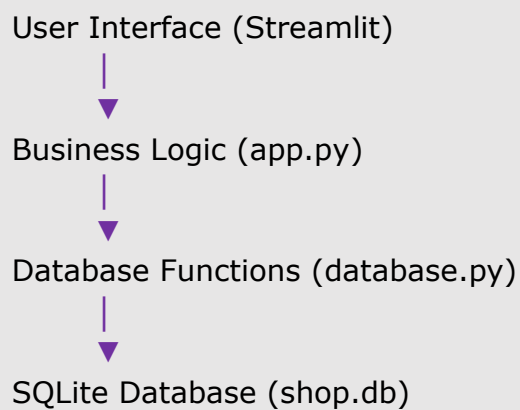
```
reset_database()
```

Deletes:

- Products
- Sales
- Reports
- Summaries

Returns system to factory state.

Overall Application Architecture





Responsibilities

Streamlit UI

- Forms
- Tables
- Charts
- Navigation

Business Logic

- Profit calculations
- Cart handling
- Validation
- Reporting

Database Layer

- CRUD operations
- Queries
- Data persistence

SQLite

- Products
- Sales
- Summaries
- Inventory records

This architecture follows a clean separation of concerns, making the system easier to maintain, debug, and extend with future features such as barcode scanning, user authentication, M-Pesa integration, PDF reporting, and cloud deployment.